

回転境界マスクパッキング

ScratchのBounding box bugを利用した
SVGアトラスの選択描画

Rotational Bounding-Box Mask Packing (RBMP)

nakakoutv

技術報告 | 2026年10月3日 | Version 1.1

公開プロジェクト : <https://scratch.mit.edu/projects/1387642307/>

概要

本報告は、Scratchの精密境界計算とSVGの描画結果の差を利用し、1枚のSVGアトラスから選択した画像のみをペンでスタンプする方式、回転境界マスクパッキング（RBMP）を提案する。小さなマーカールと整数サンプル点を避けた画像配置によって、境界計算上の凸包を線分に保つ。その回転後の軸平行外接矩形を選択領域とし、各画像の位置および事前回転を合わせて個別描画を実現する。

現行実装では、座標寸法16×8のSVGに31種類を収録した。検証時のアトラス全体のテクスチャは2048×1024px、1画像の描画解像度は最大辺28.16px相当である。日本語31文字と、黒枠・市松模様・番号からなる31画像を別々のSVGに収録し、公式配布のScratch VMおよびレンダラーを用いたローカル実行で動作を検証した。参照画像との比較では、各色成分の許容差2を超える差分画素および定義した欠落画素はいずれも0だった。

31種類は今回得られた実現例であり、方式の最大容量ではない。読み込み速度、描画時間、GPUメモリ使用量の改善は未測定である。

キーワード : Scratch、SVGアトラス、Bounding box、回転、マスク、ペンスタンプ

1. はじめに

多数の字形や画像を個別のコスチュームとして管理すると、画像資産とその読み込み処理が増える。複数画像を一つのアトラスへまとめ、必要な領域だけ表示する方法は、この資産数を減らすための候補となる。本研究では、SVGの詳細な描画と低い座標寸法に基づく境界サンプリングの差に着目した。

対象は通常のScratchプロジェクトであり、実行時のカスタム拡張や外部データ読み込みを用いない。選択情報はScratchのリストに保持し、共通カスタムブロックから参照する。文字に加えて正方形画像にも適用し、輪郭の疎な字形だけに限定されないことを検討した。

本報告の範囲は、線分状凸包を利用する選択描画、有限候補集合に対する配置探索、生成したsb3の実行および画素比較である。任意の既存アトラスから自由な矩形を選ぶ汎用切り抜きや、全連続配置に対する容量上限の証明は扱わない。

2. 線分状境界による選択描画

2.1 境界計算とSVG描画の差

本報告では、精密境界が画素の中心を用いて得られ、画素の外縁までの範囲と差が生じる挙動をBounding box bugと呼ぶ。単純な矩形では、外縁に対して各側0.5座標単位の内側となる場合がある。拡大・回転後には差も変換されるため、常に画面上の0.5pxだけ縮むという意味ではない。[1]

現行SVGのマーカーは整数座標(0,0)と(15,0)付近に置く。画像本体の外接矩形と余裕領域を整数サンプル点から避けると、詳細なテクスチャには画像が残る一方、観測される凸包集合は $C = \{(0,0), (15,0)\}$ となる。これは逐次縮小を繰り返す方式ではなく、境界計算が見落とす画像を利用して線分状の境界を構成する方式である。

2.2 回転と軸平行外接矩形

線分の長さを L 、回転角を θ 、拡大倍率を s とする。整数丸めとステージによる切り詰めを行う前の境界矩形の幅 W と高さ H は、次式で与えられる。座標系は x 右向き、 y 上向きとする。

$$W(\theta) = s L |\cos \theta|, H(\theta) = s L |\sin \theta| \quad (1)$$

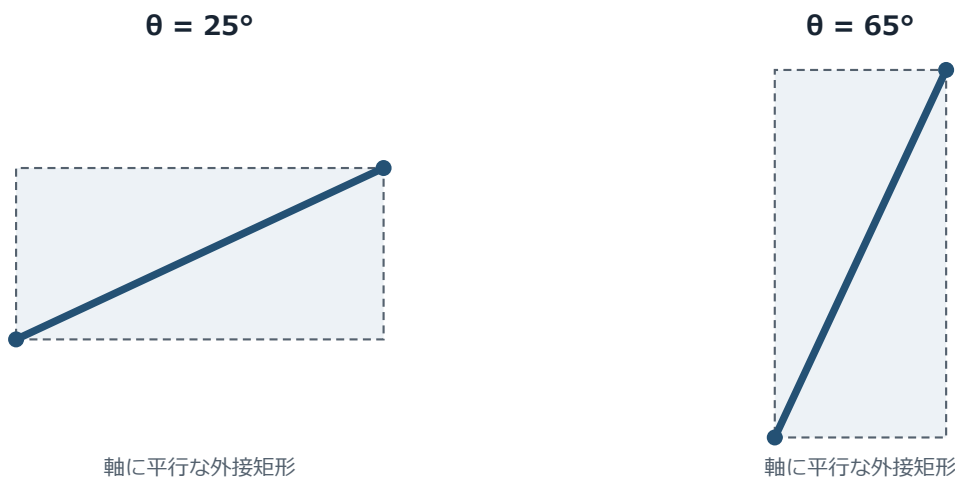


図1 同じ線分でも回転角により外接矩形の縦横比が変わる。実線は凸包、破線は選択に使う矩形を表す概念図であり、実際のアトラス配置ではない。

画像 i を選択する角度 θ_i で、その画像の外接矩形が選択領域の内側に入り、他画像の外接矩形が外側にあるよう配置する。画像自体は選択時の回転で正立するように事前回転させる。局所中心 c_i 、目標位置 t 、回転行列 R を用いると、スプライト位置は $p_i = t - s R(\theta_i) c_i$ で補正できる。SVGの y 下向き座標は生成時に変換する。

2.3 スタンプ範囲と表示サイズ

$L = 15$ 、 $\theta = 45^\circ$ では、文字側 $s = 150$ の境界は約 1591×1591 、画像側 $s = 300$ では約 3182×3182 となる。しかしスタンプ実装は境界をステージ範囲に制限して整数座標へ丸めるため、この寸法の描画バッファを毎回確保することを意味しない。[2] 実画像の表示サイズ33または66と、選択に使う境界寸法は区別する必要がある。

3. パッキング配置の探索

3.1 探索条件

各画像を最大辺dの正方形に収まるものとして扱い、角度ごとに選択矩形の角付近へ配置する。文字表示33、最低余白2を基準とした。候補角は0°と90°を除く0.1°刻みで、原SVGの範囲内への収まり、整数サンプル点の回避、他画像の除外を確認する。サンプル点の回避には外接矩形に2テクセル相当の余裕を加えた。

半幅 $h_x = L \cos \theta / 2$ 、半高さ $h_y = L \sin \theta / 2$ 、画像半辺d/2、局所余白mとすると、初期中心は選択後の座標で $(h_x - d/2 - m, -h_y + d/2 + m)$ とする。逆回転してSVG座標へ戻し、サンプル点と近すぎる場合は1テクセル刻みで内側へずらす。

3.2 適合グラフ

二つの候補を互いの選択矩形から十分に除外できるときに辺を張る。両方向の除外条件を満たすグラフの最大クリークを探索して、同時に収録できる集合を得る。候補は各角度につき採用した一配置であり、候補生成の段階で多数の連続配置を省略している。従って「最大」は生成したグラフ内に限定される。

3.3 寸法と解像度の影響

検証対象のレンダラーでは、SVGを2の累乗倍率でテクスチャ化し、最大辺の制限2048に収まる倍率を使う。
[3] 元寸法を変えるとテクスチャ倍率と整数サンプル点の位置関係の双方が変わり、容量は単調には増加しない。

表1 角付近配置・角度刻み0.1°・描画解像度28.16px・表示33・最低余白2での数値探索。31種類の実行検証は16×8について行った。

SVG座標寸法	テクスチャ倍率	全体の解像度	候補集合の容量
8×4	256	2048×1024px	29
16×8	128	2048×1024px	31
32×16	64	2048×1024px	30

同じ32×16の元寸法でも、角付近配置の刻みを0.5°から0.1°へ変更すると、候補集合の容量は29から30へ変わった。さらに16×8では31の配置が得られた。寸法を小さくしたことが増加の一般原因とは言えない。

表2 元寸法32×16、角度刻み0.1°、表示33、最低余白2で描画解像度だけを変えた数値探索。各候補のsb3実行は本報告の評価対象外。

最大辺の描画解像度	候補集合の容量
24px	38
28.16px	30
32px	27
40px	21

31種類は現行パラメーターで得られた実現例である。解像度、余白、画像の縦横比、候補生成方法を変えた場合の容量上限は別の問題として残る。

4. Scratchプロジェクトの実装

表3 現行実装のパラメーター。SVG寸法はベクトル座標、表示サイズはステージ座標、解像度はテクスチャ上の画素数相当を示す。

項目	文字スプライト	番号付き画像スプライト
収録数 / パッキングSVG数	31種類 / 1枚	31種類 / 1枚
元SVGの座標寸法	16×8	16×8
画像のSVG内最大辺	0.22	0.22
全体のテクスチャ解像度	2048×1024px	2048×1024px
1画像の描画解像度	最大辺28.16px相当	最大辺28.16px相当
表示サイズ / 拡大倍率	最大辺33 / 150倍	66×66 / 300倍
Scratchのサイズ値	15000%	30000%
選択方法	1秒ごとの順送り	画像番号スライダー1～31

描画解像度 $q = dT = 0.22 \times 128 = 28.16$ 、表示サイズ $a = ds$ である。番号付き画像を66へ拡大しても q は増えない。整数の28画素四方の独立画像があるという意味ではなく、回転前の局所最大辺に対するテクスチャ画素数の換算値である。

4.1 リスト参照式の共通描画

各スプライトに「画像ID」「描画X補正」「描画Y補正」「描画方向」「描画サイズ」「描画コスチューム」の6リストを用意し、同じ行を一画像のレコードとする。共通ブロック「画像を描画 (画像ID) x (x) y (y)」はIDを検索し、未登録なら何も描画しない。

共通描画ブロック（画面を再描画せずに実行）：

- IDを検索し、その行の位置補正・方向・サイズを参照
- スプライトを表示 → 空コスチュームへ切り替え
- サイズを1 / 0に設定 → 補正した位置へ移動
- 方向を設定 → リストの有限サイズへ戻す
- パッキングしたSVGへ切り替え
- もし〈端に触れた〉なら：何もしない
- スタンプ → スプライトを隠す

空コスチュームと無限大のサイズは、有限サイズに戻す前の移動制限回避に用いる。空のifでは条件の評価だけを行い、精密な境界計算を発生させる。[4] 吹き出し表示による境界計算は使用しない。各スプライトにはパッキングSVGに加えて空コスチュームが一つある。

4.2 共有するペン描画面

二つのスプライトは同じペン描画面にスタンプする。ステージが文字の切り替え時刻またはスライダー値の変化を待ち、ペンを消去して両スプライトへ描画を通知する。これにより片方だけの消去で他方が失われることを避け、変更のない間の再スタンプを省く。

5. 実行検証と画素比較

5.1 評価環境と比較対象

公式配布のscratch-render、scratch-vm、scratch-storageをローカルHTMLから読み込み、ヘッドレスChromeとPlaywrightで生成sb3を実行した。ステージは480×360、deviceScaleFactorは1とした。実行スクリプトはSwiftShaderの使用を許可する起動引数を設定したが、物理GPUの速度評価は行っていない。公式ScratchサイトのエディターUIでの実行確認ではない。

参照画像は、同じSVGから目的の画像グループだけを取り出し、境界マーカーを除いて同じ座標寸法・回転中心・方向・サイズ・位置で通常描画した画像である。文字は文字単体の31文字版でステージ全体を比較した。番号付き画像は二スプライト版の右側100×100px領域を比較し、左側の文字も残ることを確認した。

5.2 差分の定義

RGB各成分の絶対差の最大値が2を超える画素を差分画素とした。欠落画素は、参照側のRGB最小成分が245未満で、出力側のRGB全成分が250を超える画素と定義した。これは同じラスタライズ条件の参照との一致を評価するものであり、理想的なベクトル輪郭からの画質劣化を測る指標ではない。

表4 各データセット31種類の全件検証。値は各画像について得られた値の最大値。

評価項目	文字31種類	番号付き画像31種類
許容差を超える差分画素	0	0
定義した欠落画素	0	0
観測された凸包	[(0,0),(15,0)]	[(0,0),(15,0)]

5.3 状態変化の検証

文字31種類の順序と末尾から先頭への復帰、スライダー変数1〜31の整数設定と全値の選択、二スプライトの共存を確認した。スライダー検証はVM内の対応変数へ値を書き込み、描画結果とモニター設定を確認する方法であり、ブラウザーUIのつまみ操作そのものは試験していない。さらにリスト行の入れ替え、補正值変更、任意ID、未登録ID、x・y引数を検証し、描画がリスト参照に従うことを確認した。



図2 文字31種類の描画結果。左上の英数字は検証画像を整理するための識別ラベルであり、現行デモのキー操作を表すものではない。表示サイズは文字の最大辺33。

6. 正方形画像への適用と考察



図3 黒枠、白と灰色の6×6市松模様、番号1～31を重ねた31種類。別々のコスチュームではなく一つのSVGに収録した画像を選択してスタンプした結果を並べている。表示66×66、描画解像度最大辺28.16px相当。

6.1 画像の種類と画質

正方形の全面を使う画像でも、整数サンプル点を避ける配置と選択範囲への収まりを満たせば、線分状の凸包を保って描画できた。画像は黒い枠を含む 0.22×0.22 の局所領域内に収め、文字で得られた31候補の中心と角度を再利用した。番号はMeiryoの輪郭をSVGパスへ変換して収録し、実行時にフォントを読み込まない。

回転前後のラスタライズと補間に加え、 $66 / 28.16 \approx 2.34$ 倍の拡大により、枠線や数字には粗さが見える。参照との差分0は切り抜きによる欠けが検出されなかったことを示すが、輪郭が高精細であることを示さない。

6.2 処理負荷と適用範囲

資産数を減らす目的で二つのアトラスを用意したが、大きなテクスチャと境界計算にもコストがある。選択画像が小さくても境界は広く、ステージとの交差領域には透明部分を含み得る。コスチューム数の減少だけから読み込み・描画・メモリの改善を断定できない。個別コスチュームとの比較測定が今後の課題である。

方式は境界計算とスタンプの実装に依存する。レンダラーの変更、描画環境や表示倍率の差による結果は未評価である。また、あらかじめ選択角度と配置を合わせた専用アトラスを必要とし、任意の既存アトラスにそのまま適用できるものではない。

6.3 結論

RBMPによって、 16×8 のSVG一枚あたり31種類の選択スタンプを実現した。文字と正方形画像の両方について、同じ描画条件の単独参照と比較し、定義した差分画素・欠落画素が0となる実現例を得た。今後は解像度と容量のトレードオフ、配置探索の拡張、および速度・メモリの定量評価を行う。

7. 再現性と資料

本報告は2026年10月3日時点の作業フォルダ内の成果物と検証記録に基づく。ブラウザー実行を伴う検証スクリプトは、この環境のランタイムパスを使用している。再現時は自分の環境に合わせてNode.js、PlaywrightおよびChromeのパスを調整する必要がある。

使用環境：Windows、Node.js 24.19.0、Chrome 154.0.8037.58、Python 3.12.14。PDF作成にはReportLab 4.4.9を使用した。これらは資料作成時に確認した環境値であり、他環境での同一結果を保証するものではない。

7.1 生成と検証

```
python 生成/build_dense_experiment.py --optimized
PowerShellで 生成/export_numbers.ps1 を実行
python 生成/build_two_sprite_demo.py
node 検証/verify-optimized.cjs
python 検証/inspect_optimized.py
node 検証/verify-list-data.cjs
node 検証/verify-two-sprite.cjs
python 検証/inspect_numbered_tiles.py
```

Pillowを含むPython環境を使用する。文字参照画像の再作成には検証/verify-unmasked.cjsを使用する。番号付き画像の参照はverify-two-sprite.cjsが生成する。数値探索は生成/search_packing_combinations.pyおよびrefine_packing_search.pyに収録した。

設定ファイル：素材/optimized-config.json、素材/numbered-tiles-config.json。評価記録：検証結果/データ/optimized-pixel-check.json、numbered-tiles-pixel-check.json、two-sprite-verification.json、list-data-verification.json。設定には31画像分の選択角度・位置補正値が含まれる。

7.2 検証用配布ライブラリの識別

```
scratch-render.js / SHA-256
fba1958d3d3d992c9dc1f9e06b7d1ed0500f386508ccf23aeaf27390bc02a688

scratch-vm.js / SHA-256
749129c753a9d6fd48ecbcde879cadda74f97dbd226607307c6c2d5a5b0d9fa

scratch-storage.js / SHA-256
4fd168a2b4fa0e11e6d3f9ab0b4f72b1751fb9d1e7df0bca02f4ed7871e87f22
```

配布ライブラリのバージョンを一意に示すため、同梱ファイルのハッシュ値を記載した。以下のdevelopブランチのソースは実装説明の参照先であり、同梱ビルドと完全に同一のリビジョンであることは主張しない。

参考文献・一次資料

- [1] Scratch Foundation. scratch-render, Drawable.js. 精密境界と凸包座標の変換。
<https://github.com/scratchfoundation/scratch-render/blob/develop/src/Drawable.js>
- [2] Scratch Foundation. scratch-render, RenderWebGL.js. penStampおよび_touchingBounds。
<https://github.com/scratchfoundation/scratch-render/blob/develop/src/RenderWebGL.js>
- [3] Scratch Foundation. scratch-render, SVGSkin.js. SVGのテクスチャ化と最大寸法。
<https://github.com/scratchfoundation/scratch-render/blob/develop/src/SVGSkin.js>
- [4] Scratch Foundation. scratch-vm, scratch3_sensing.js. touchingObjectの呼び出し。
https://github.com/scratchfoundation/scratch-vm/blob/develop/src/blocks/scratch3_sensing.js
- [5] nakakoutv. RBMPデモプロジェクト. Scratch.
<https://scratch.mit.edu/projects/1387642307/>

一次資料の参照日：2026年10月3日。参照画像と実測値は本研究の作業フォルダ内の記録による。

付録A. 生成した単一SVGの全体図

生成したSVGアトラスの実配置を図4と図5に示す。どちらも座標寸法16×8の一つのSVGで、31種類の画像を位置と角度を変えて収録している。透明部分を白で表示し、原SVGを128倍の2048×1024pxに描画した。小さな境界用マーカーも原SVGのまま含めている。

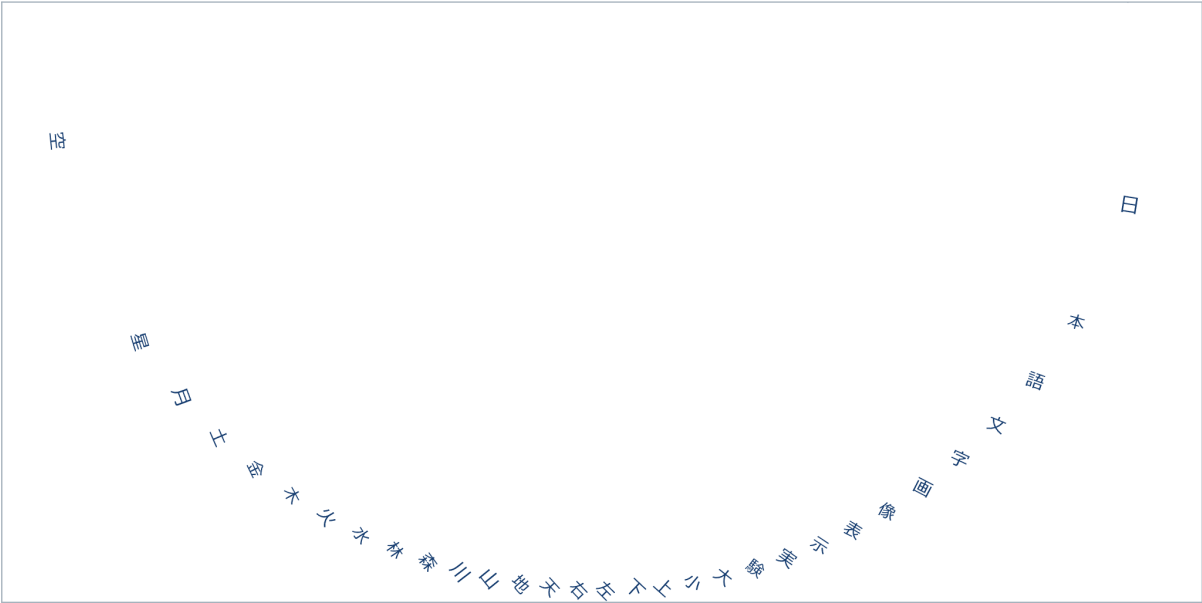


図4 日本語31文字を収録した単一SVGの全体。字形の位置と事前回転が、各文字を選択する角度に対応する。原SVG : packed-font-optimized.svg。

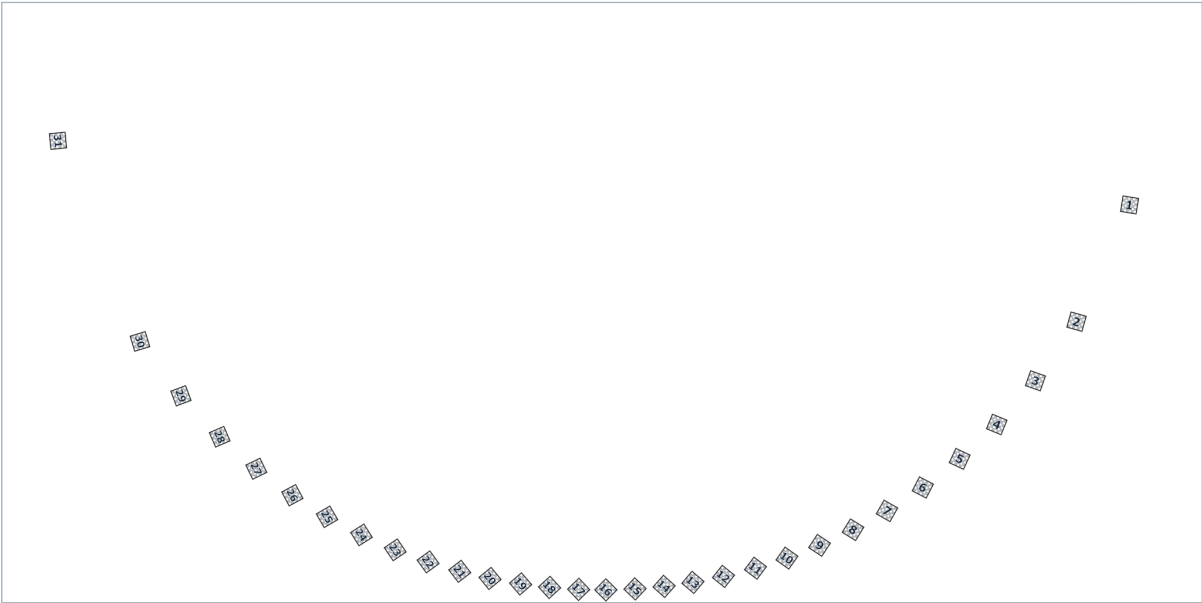


図5 番号付き画像31種類を収録した単一SVGの全体。各画像の黒枠・市松模様・番号をまとめて配置している。原SVG : packed-numbered-tiles.svg。

この全体図は、SVGの座標領域と配置を示すための図である。図2・図3の描画結果は、各SVGを所定の方向と位置でスタンプして得られる。PDFの閲覧時に拡大すると、個々の字形や番号の配置を確認できる。